

Inicialización de Objetos y Gestión de Memoria en C++

Asignatura: Programación Avanzada y Algoritmos (LIIS2105) | **Semestre:** Tercero

Docente: Mtro. Vázquez Mora Paulo Daniel | **Institución:** Universidad Iberoamericana Puebla

Objetivo de la Práctica: Esta actividad guiada tiene como propósito que el estudiante comprenda y aplique de manera práctica los mecanismos de inicialización de objetos en C++ mediante constructores. Se analizará la importancia de las listas de inicialización para atributos constantes y la diferencia fundamental en memoria entre la copia superficial (shallow copy) y la copia profunda (deep copy) de objetos con punteros dinámicos.

Ejercicio 1: El Desafío del Sensor (Constantes y Sobrecarga)

Fundamento Teórico: Un constructor inicializa un objeto de tipo clase garantizando un estado coherente. En C++, los atributos declarados como constantes (const) o de tipo referencia **no pueden ser asignados** dentro del cuerpo del constructor mediante el operador de igualdad (=). La única forma válida de inicializarlos es antes de que se ejecute el cuerpo del constructor, utilizando una **lista de inicialización de miembros**. Esta lista se coloca inmediatamente después de los parámetros del constructor precedida por el operador de dos puntos (:). Adicionalmente, se puede emplear la *sobrecarga de constructores* para definir múltiples formas de crear un objeto.

Instrucciones de la Práctica:

- Analiza el código base proporcionado abajo. Observa que el atributo id es constante y privado.
- Intenta compilar inicialmente el programa. Si modificas el constructor parametrizado para inicializar el ID haciendo `id = nuevo_id;` dentro de las llaves {}, describe qué error arroja el compilador.
- Corrige la inicialización utilizando la lista de inicialización de miembros para que el código compile correctamente.
- Discute por qué es de buena práctica declarar como const los métodos de consulta como `mostrarEstado()`.

```
#include <iostream>
using namespace std;

class Sensor {
private:
    const int id;        // Atributo constante (obligatorio usar lista de inicialización)
    float lectura;

public:
    // 1. Constructor por defecto
    Sensor() : id(0), lectura(0.0f) {
        cout << "Constructor por defecto: Sensor ID " << id << " inicializado." << endl;
    }

    // 2. Constructor parametrizado (Corrige la inicialización de constantes)
    Sensor(int nuevo_id, float lectura_inicial) : id(nuevo_id), lectura(lectura_inicial) {
        cout << "Constructor parametrizado: Sensor ID " << id << " creado." << endl;
    }

    void actualizarLectura(float nueva_lectura) {
        lectura = nueva_lectura;
    }

    // Método constante (de sólo lectura)
    void mostrarEstado() const {
        cout << "Sensor [" << id << "] - Lectura: " << lectura << " C" << endl;
    }
};

int main() {
    Sensor s1;                // Invocación del constructor por defecto
    Sensor s2(101, 24.5f);    // Invocación del constructor parametrizado

    s1.mostrarEstado();
    s2.mostrarEstado();

    // s2.id = 102; // Descomenta esta línea: ¿Qué error de compilación obtienes?
    return 0;
}
```

Preguntas de Control para el Alumno:

- **Pregunta 1.1:** ¿Por qué no es posible inicializar la variable `id` mediante asignación ordinaria en el cuerpo del constructor?
- **Pregunta 1.2:** Si una clase tiene miembros constantes y referencias, ¿cuál es el único mecanismo permitido para inicializarlos?
- **Pregunta 1.3:** Explica qué sucede en memoria cuando se invoca un constructor parametrizado frente a uno por defecto.

Ejercicio 2: El Peligro de Compartir Punteros (Copia Superficial vs. Profunda)

Fundamento Teórico: Por defecto, C++ genera un *constructor de copia por defecto* si el programador no define uno. Este constructor realiza una **copia superficial (shallow copy)**, duplicando los valores de los miembros bit a bit. Si la clase administra memoria dinámica (punteros creados con `new`), la copia superficial duplica la dirección de memoria guardada en el puntero, no los datos reales. Esto provoca que ambos objetos apunten al mismo bloque de memoria en el Heap. Cuando uno de los objetos modifica los datos, altera también al otro. Peor aún, al destruirse los objetos, ambos intentarán liberar la misma dirección de memoria usando el operador `delete` en su destructor, resultando en un fallo crítico de ejecución conocido como **Double Free**. Para solucionar esto, es obligatorio escribir un constructor de copia personalizado que reserve un bloque de memoria nuevo y copie los datos reales (**copia profunda o deep copy**).

Instrucciones de la Práctica:

- Compila y ejecuta el código base que se presenta abajo (el cual carece de constructor de copia personalizado).
- Observa el comportamiento de las colecciones `col1` y `col2` tras modificar un valor de la segunda.
- Analiza el resultado en la consola al finalizar la ejecución del programa. ¿Ocurre un error de violación de segmento o caída del programa? ¿Qué significa?
- Descomenta el bloque del constructor de copia personalizado para implementar la copia profunda y vuelve a ejecutar. Describe las diferencias observadas.

```

#include <iostream>
using namespace std;

class ColeccionDatos {
private:
    int* datos;
    int capacidad;

public:
    // Constructor parametrizado (reserva memoria dinámica)
    ColeccionDatos(int tam) : capacidad(tam) {
        datos = new int[capacidad]; // Asignación de memoria en el Heap
        for (int i = 0; i < capacidad; i++) datos[i] = 0;
        cout << "Memoria de " << capacidad << " enteros reservada en " << (void*)datos << endl;
    }

    // Destructor (libera memoria en Heap)
    ~ColeccionDatos() {
        delete[] datos; // Libera la memoria asignada dinámicamente
        cout << "Memoria liberada correctamente de " << (void*)datos << endl;
    }

    // [INSTRUCCIÓN]: Descomenta este constructor para habilitar la COPIA PROFUNDA
    /*
    ColeccionDatos(const ColeccionDatos& otra) : capacidad(otra.capacidad) {
        datos = new int[capacidad]; // Solicita un bloque de memoria nuevo e independiente
        for (int i = 0; i < capacidad; i++) {
            datos[i] = otra.datos[i]; // Copia el contenido real de los elementos
        }
        cout << "Constructor de copia (Copia Profunda) en " << (void*)datos << endl;
    }
    */

    void modificarDato(int indice, int valor) {
        if (indice >= 0 && indice < capacidad) datos[indice] = valor;
    }

    void imprimir() const {
        for (int i = 0; i < capacidad; i++) cout << datos[i] << " ";
        cout << endl;
    }
};

int main() {
    cout << "--- Creando Coleccion 1 ---" << endl;
    ColeccionDatos col1(5);
    col1.modificarDato(0, 99);

    cout << "\n--- Creando Coleccion 2 como copia de 1 ---" << endl;
    ColeccionDatos col2 = col1; // Constructor de copia por defecto (Copia Superficial)

    cout << "Col1: "; col1.imprimir();
    cout << "Col2: "; col2.imprimir();

    cout << "\n--- Modificando Col2 ---" << endl;
    col2.modificarDato(0, 50);

    cout << "Col1 (despues de cambiar Col2): "; col1.imprimir();
    cout << "Col2 (despues de cambiar Col2): "; col2.imprimir();

    cout << "\n--- Fin del programa (Invocando destructores) ---" << endl;
    return 0;
}

```

Preguntas de Control para el Alumno:

- **Pregunta 2.1:** ¿Por qué ambas variables impresas en consola muestran el valor 50 tras modificar únicamente col2 en copia superficial?
- **Pregunta 2.2:** Explica qué es un error de 'Double Free' y bajo qué condiciones de gestión de memoria dinámica ocurre.

- **Pregunta 2.3:** ¿Cómo soluciona el constructor de copia profunda el error de inestabilidad y compartición de punteros en memoria?

Guía y Notas Pedagógicas para el Docente

Notas de Laboratorio:

- **Ejercicio 1:** El estudiante comprenderá que la declaración `const int id` define un atributo inmutable. Si intentan inicializarlo dentro del cuerpo `{ id = nuevo_id; }`, el compilador genera un error indicando que una variable constante no puede ser asignada después de su construcción. Esto aclara la distinción conceptual entre *inicialización* (lista de miembros) y *asignación* (cuerpo del constructor).
- **Ejercicio 2:** Ejecutar el código sin el constructor de copia profunda personalizado ilustra de manera directa cómo ambos objetos apuntan a la misma dirección de memoria dinámica. El docente debe guiar a los alumnos a examinar las direcciones de puntero mostradas en consola (por ejemplo, `0x5555...`) y notar que son idénticas en copia superficial, pero distintas en copia profunda. El fallo por Double Free será reportado por el sistema operativo al destruir `col1` y `col2` consecutivamente al final de `main()`.